

第一部分：理论知识

1.1 为什么需要用户管理？

生活化类比：公司门禁系统

想象一下你们公司的门禁系统：

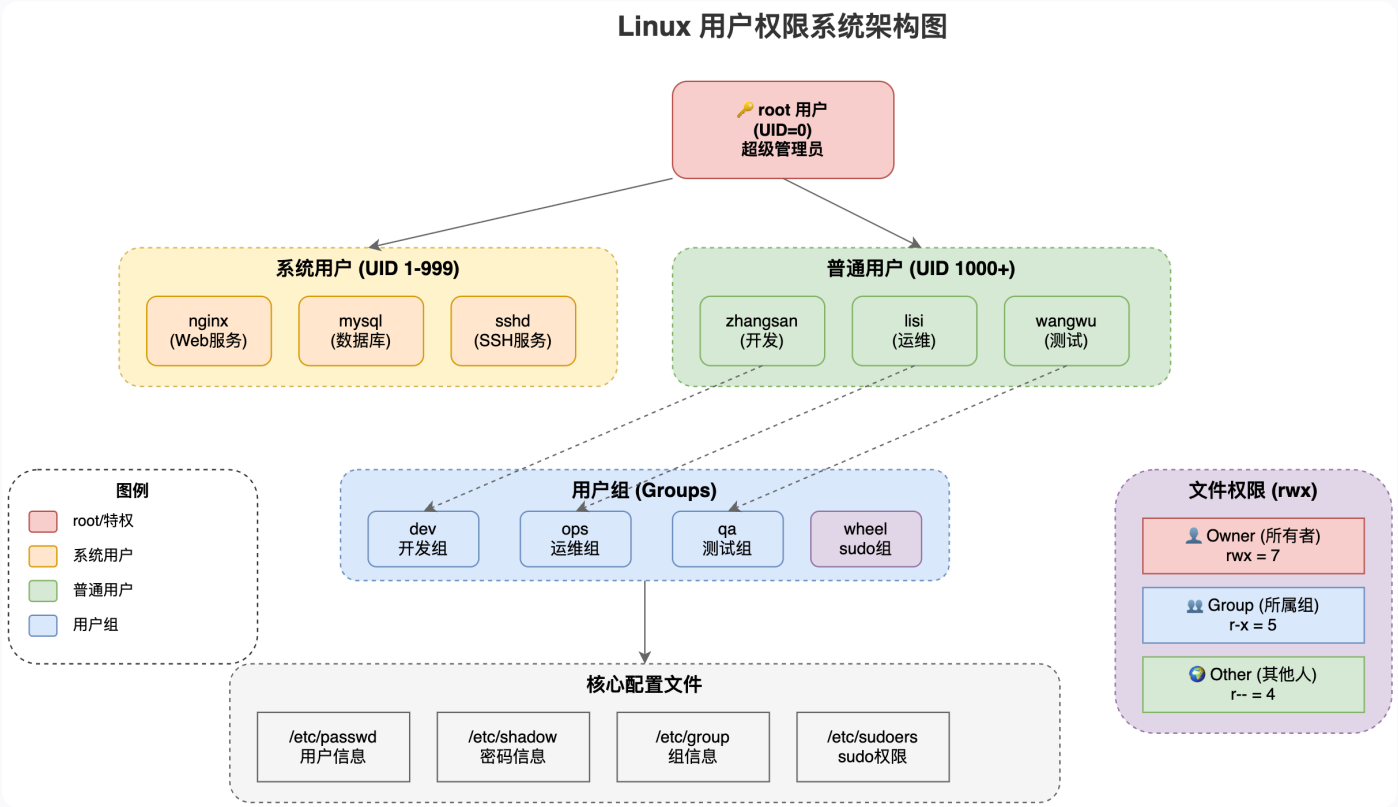
- 普通员工：只能刷卡进入自己的办公区域
- 部门经理：可以进入本部门所有房间
- CEO：可以进入公司任何地方，包括机房、财务室
- 访客：只能在前台等候区活动

Linux 的用户管理就像这个门禁系统，不同的人（用户）有不同的权限，能访问的资源也不同。

为什么要这样设计？

目的	说明	生活类比
安全性	防止未授权访问敏感数据	银行金库不是谁都能进
隔离性	用户之间互不干扰	每个人有自己的储物柜
可追溯	记录谁做了什么操作	门禁刷卡记录
资源管理	限制用户使用的系统资源	每人每月流量限额

1.2 Linux 用户的三种类型



生活化类比：酒店住客

用户类型	UID 范围	类比	说明
root 用户	0	酒店老板	拥有最高权限，可以做任何事
系统用户	1-999	酒店工作人员	运行服务程序，一般不能登录
普通用户	1000+	酒店住客	日常使用的账户，权限受限

1.3 用户和用户组的关系

生活化类比：学校班级

- 用户 = 学生
- 用户组 = 班级
- 一个学生必须属于至少一个班级（主组）

- 一个学生可以参加多个社团 (附加组)
- 一个班级可以有很多学生

1.4 Linux 权限系统详解

生活化类比：文件柜权限

想象一个文件柜，你对它可以有三种操作：

权限	符号	数字	对文件的含义	对目录的含义	生活类比
读	r	4	查看文件内容	列出目录内容	能打开柜子看里面有什么
写	w	2	修改文件内容	在目录中创建/删除文件	能往柜子里放东西或拿走
执行	x	1	运行程序/脚本	进入目录	能打开柜门走进去

权限的三个层次

每个文件/目录对三类人分别设置权限：

```
-rwxr-xr-- 1 root dev 4096 Jan 1 10:00 myfile.txt
```


权限数字计算

```
rwX = 4 + 2 + 1 = 7
r-x = 4 + 0 + 1 = 5
r-- = 4 + 0 + 0 = 4

所以 rwxr-xr-- = 754
```

常用权限组合速查

数字权限	符号权限	含义	使用场景
755	rwxr-xr-x	所有者完全控制，其他人可读可执行	可执行程序、脚本
644	rw-r--r--	所有者可读写，其他人只读	普通配置文件
600	rw-----	仅所有者可读写	敏感文件如私钥
700	rwx-----	仅所有者完全控制	私人目录
777	rwxrwxrwx	所有人完全控制	⚠ 危险！ 尽量避免

第二部分： 实践操作

2.1 实验一： 用户管理基础操作

实验目标

掌握用户的创建、修改、删除等基本操作。

实验步骤

步骤 1： 查看当前用户信息

```
# 查看当前登录用户
whoami

# 查看当前用户的详细信息
id

# 查看当前登录的所有用户
who

# 查看用户登录历史
last | head -20
```

预期输出示例：

```
[root@localhost ~]# whoami
root

[root@localhost ~]# id
uid=0(root) gid=0(root) groups=0(root)
```

步骤 2：创建新用户

```
# 创建用户 zhangsan（自动创建家目录）
useradd zhangsan

# 验证用户是否创建成功
id zhangsan

# 查看 /etc/passwd 中的用户信息
grep zhangsan /etc/passwd

# 查看用户家目录是否创建
ls -la /home/zhangsan
```

预期输出示例：

```
[root@localhost ~]# id zhangsan
uid=1001(zhangsan) gid=1001(zhangsan) groups=1001(zhangsan)
```

步骤 3：创建用户的高级选项

```
# 创建系统用户（不创建家目录，不能登录）
useradd -r -s /sbin/nologin nginx_user

# 创建用户时指定主组
useradd -g dev wangwu

# 创建用户时指定附加组
useradd -G wheel,docker liuqi
```

命令参数速查表：

参数	说明	示例
<code>-u UID</code>	指定用户 ID	<code>-u 1005</code>
<code>-g GROUP</code>	指定主组	<code>-g dev</code>
<code>-G GROUPS</code>	指定附加组（多个用逗号分隔）	<code>-G wheel,docker</code>
<code>-d HOME</code>	指定家目录	<code>-d /home/zhangsan</code>
<code>-M</code>	不创建家目录	用于系统用户
<code>-c COMMENT</code>	用户描述	<code>-c "开发工程师"</code>
<code>-r</code>	创建系统用户	用于服务账户

步骤 4：设置用户密码

```
# 为 zhangsan 设置密码（交互式）
passwd zhangsan

# 使用非交互式设置密码（脚本中使用）
echo "MyP@ssw0rd123" | passwd --stdin zhangsan      # CentOS/RHEL
# 或者
echo "zhangsan:MyP@ssw0rd123" | chpasswd            # 通用方式

# 查看密码状态
passwd -S zhangsan
```

预期输出示例：

```
[root@localhost ~]# passwd zhangsan
Changing password for user zhangsan.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.

[root@localhost ~]# passwd -S zhangsan
zhangsan PS 2024-01-01 0 99999 7 -1 (Password set, SHA512 crypt.)
```

步骤 5：修改用户信息

```
# 修改用户描述
usermod -c "高级开发工程师" zhangsan

# 修改用户的登录 Shell
usermod -s /bin/zsh zhangsan

# 将用户添加到附加组（追加方式，保留原有组）
usermod -aG wheel zhangsan
usermod -aG docker zhangsan

# 修改用户名
usermod -l zhangsan_new zhangsan

# 锁定用户账户（禁止登录）
```

```
usermod -L zhangsan

# 解锁用户账户
usermod -U zhangsan

# 设置账户过期日期
usermod -e 2024-12-31 zhangsan
```

⚠ 重要提示:

- 使用 `-aG` 时的 `-a` 参数非常重要, 表示"追加"
- 如果只用 `-G` 不加 `-a`, 会覆盖用户的所有附加组!

步骤 6: 删除用户

```
# 查看要删除的用户信息
id lisi
ls -la /home/lisi

# 删除用户 (保留家目录)
userdel lisi

# 删除用户及其家目录和邮箱
userdel -r wangwu

# 验证删除结果
id lisi          # 应该报错: no such user
ls -la /home/lisi # 如果用 userdel (不带-r) , 目录还在
```

✅ 实验一检查点

完成以下任务来验证你的学习成果:

```
# 任务1: 创建用户 testuser1, UID 为 2001, 描述为 "测试用户1"
useradd -u 2001 -c "测试用户1" testuser1

# 任务2: 为 testuser1 设置密码为 Test@123
echo "Test@123" | passwd --stdin testuser1
```



```
# 任务3: 查看 testuser1 的完整信息
id testuser1
grep testuser1 /etc/passwd

# 任务4: 将 testuser1 添加到 wheel 组
usermod -aG wheel testuser1

# 任务5: 验证 testuser1 是否在 wheel 组中
groups testuser1

# 清理: 删除测试用户
userdel -r testuser1
```

2.2 实验二：用户组管理

实验目标

掌握用户组的创建、修改、删除及成员管理。

实验步骤

步骤 1：查看组信息

```
# 查看当前用户所属的组
groups

# 查看指定用户所属的组
groups root

# 查看所有组
cat /etc/group

# 查看特定组信息
grep wheel /etc/group
```

步骤 2：创建用户组

```
# 创建普通用户组
groupadd dev
groupadd ops
groupadd qa

# 创建指定 GID 的组
groupadd -g 3000 dba

# 创建系统组
groupadd -r nginx

# 验证创建结果
tail -5 /etc/group
```

步骤 3：修改用户组

```
# 修改组名
groupmod -n developers dev

# 修改组 GID
groupmod -g 3001 dba

# 验证修改
grep developers /etc/group
grep dba /etc/group
```

步骤 4：用户组成员管理

```
# 先创建测试用户
useradd user1
useradd user2
useradd user3

# 方式一：创建用户时指定组
useradd -g developers -G ops,qa newuser
```

```
# 方式二：将已有用户添加到组
usermod -aG developers user1
usermod -aG developers user2
usermod -aG ops user3

# 方式三：使用 gpasswd 管理组成员
gpasswd -a user3 developers    # 将 user3 添加到 developers 组
gpasswd -d user3 developers    # 将 user3 从 developers 组移除

# 查看组成员
grep developers /etc/group
getent group developers
```

步骤 5：删除用户组

```
# 删除空组
groupdel qa

# 注意：不能删除用户的主组
# 如果组是某用户的主组，需要先修改该用户的主组或删除用户
groupdel developers    # 如果有用户主组是 developers，会失败

# 验证删除
grep qa /etc/group
```



组管理命令速查表

命令	功能	示例
<code>groupadd</code>	创建组	<code>groupadd dev</code>
<code>groupmod</code>	修改组	<code>groupmod -n newname oldname</code>
<code>groupdel</code>	删除组	<code>groupdel groupname</code>
<code>gpasswd -a</code>	添加用户到组	<code>gpasswd -a user group</code>
<code>gpasswd -d</code>	从组删除用户	<code>gpasswd -d user group</code>
<code>groups</code>	查看用户所属组	<code>groups username</code>
<code>getent group</code>	查看组信息	<code>getent group groupname</code>

2.3 实验三：文件权限管理

实验目标

掌握文件和目录权限的查看与修改。

实验步骤

步骤 1：创建实验环境

```
# 创建实验目录
mkdir -p /opt/permission-lab
cd /opt/permission-lab

# 创建测试文件和目录
touch file1.txt file2.txt script.sh
mkdir dir1 dir2

# 写入一些内容
echo "This is file1" > file1.txt
echo "This is file2" > file2.txt
```

```
echo '#!/bin/bash' > script.sh
echo 'echo "Hello World"' >> script.sh

# 查看创建的文件
ls -la
```

步骤 2：理解权限显示

```
# 查看详细权限
ls -la

# 输出解读
# -rw-r--r-- 1 root root 14 Jan 1 10:00 file1.txt
# drwxr-xr-x 2 root root 4096 Jan 1 10:00 dir1
```

权限字段分解：

```
-rw-r--r--
| | | |
| | | | 其他人权限：r--（只读）
| | | | 所属组权限：r--（只读）
| | | | 所有者权限：rw-（读写）
| | | | 文件类型：-（普通文件）或 d（目录）
```

步骤 3：使用 chmod 修改权限（符号模式）

```
# 符号模式语法：chmod [ugoa][+ -=][rwx] filename
# u=user(所有者), g=group(组), o=other(其他), a=all(所有)
# + 添加权限, - 移除权限, = 设置权限

# 给所有者添加执行权限
chmod u+x script.sh
ls -l script.sh

# 给组添加写权限
chmod g+w file1.txt
ls -l file1.txt
```

```
# 移除其他人的读权限
chmod o-r file2.txt
ls -l file2.txt

# 给所有人添加执行权限
chmod a+x script.sh

# 设置精确权限
chmod u=rwx,g=rx,o=r script.sh
ls -l script.sh
```

步骤 4：使用 chmod 修改权限（数字模式）

```
# 数字模式：r=4，w=2，x=1

# 设置权限为 755 (rwxr-xr-x)
chmod 755 script.sh
ls -l script.sh

# 设置权限为 644 (rw-r--r--)
chmod 644 file1.txt
ls -l file1.txt

# 设置权限为 600 (rw-----)
chmod 600 file2.txt
ls -l file2.txt

# 设置目录权限为 700
chmod 700 dir1
ls -ld dir1
```

步骤 5：递归修改权限

```
# 创建嵌套目录结构
mkdir -p dir2/subdir1/subdir2
touch dir2/subdir1/file1.txt
touch dir2/subdir1/subdir2/file2.txt

# 递归修改目录及其内容的权限
chmod -R 755 dir2

# 验证
ls -laR dir2
```

步骤 6：修改文件所有者和所属组

```
# 创建测试用户和组
useradd testuser
groupadd testgroup

# 修改文件所有者
chown testuser file1.txt
ls -l file1.txt

# 修改文件所属组
chgrp testgroup file1.txt
ls -l file1.txt

# 同时修改所有者和组
chown testuser:testgroup file2.txt
ls -l file2.txt

# 只修改组（使用 chown）
chown :testgroup script.sh
ls -l script.sh

# 递归修改目录及其内容
chown -R testuser:testgroup dir1
ls -laR dir1
```



权限命令速查表

命令	功能	示例
<code>chmod</code>	修改权限	<code>chmod 755 file</code>
<code>chown</code>	修改所有者	<code>chown user file</code>
<code>chgrp</code>	修改所属组	<code>chgrp group file</code>
<code>chown user:group</code>	同时修改	<code>chown user:group file</code>
<code>-R</code>	递归操作	<code>chmod -R 755 dir</code>

实验三检查点

```
# 创建文件并设置以下权限
touch /tmp/test_perm.txt

# 任务1: 设置权限为 所有者读写, 组只读, 其他人无权限 (640)
chmod 640 /tmp/test_perm.txt
ls -l /tmp/test_perm.txt

# 任务2: 将文件所有者改为 testuser
chown testuser /tmp/test_perm.txt
ls -l /tmp/test_perm.txt

# 任务3: 使用符号模式给其他人添加读权限
chmod o+r /tmp/test_perm.txt
ls -l /tmp/test_perm.txt

# 清理
rm /tmp/test_perm.txt
```

2.4 实验五：sudo 权限配置

实验目标

掌握 `sudo` 的配置和使用，实现精细化权限控制。



实验步骤

步骤 1：理解 `sudo`

```
# sudo 让普通用户以其他用户（通常是 root）身份执行命令
# 配置文件：/etc/sudoers

# 查看 sudoers 文件（只读）
cat /etc/sudoers

# 正确的编辑方式（带语法检查）
visudo
```

步骤 2：`sudoers` 文件格式

```
# sudoers 文件格式：
# 用户/组      主机=(以谁的身份)      可执行的命令

# 示例解读：
# root      ALL=(ALL)          ALL
# |          |      |          |
# |          |      |          |—— 可执行所有命令
# |          |      |—— 可以以任何用户身份
# |          |—— 在所有主机上
# |—— root 用户

# %wheel    ALL=(ALL)          ALL
# %wheel 表示 wheel 组的所有成员
```

步骤 3：常用 `sudo` 配置示例

```
# 使用 visudo 编辑配置
visudo
```

```
# 示例1: 允许 zhangsan 执行所有命令
zhangsan    ALL=(ALL)        ALL

# 示例2: 允许 zhangsan 执行所有命令, 无需密码
zhangsan    ALL=(ALL)        NOPASSWD: ALL

# 示例3: 允许 ops 组成员重启服务
%ops        ALL=(ALL)        /usr/bin/systemctl restart *,
/usr/bin/systemctl status *

# 示例4: 允许 dev 组成员查看日志
%dev        ALL=(ALL)        /usr/bin/tail -f /var/log/*, /usr/bin/cat
/var/log/*
```

步骤 4: 实践 sudo 配置

```
# 创建测试用户
useradd sudotest
passwd sudotest

# 方法1: 将用户加入 wheel 组 (推荐)
usermod -aG wheel sudotest

# 验证
su - sudotest
sudo whoami    # 输入 sudotest 的密码, 应该返回 root
exit

# 方法2: 在 /etc/sudoers.d/ 创建独立配置文件 (推荐)
cat > /etc/sudoers.d/sudotest << 'EOF'
# 允许 sudotest 执行特定命令
sudotest ALL=(ALL) NOPASSWD: /usr/bin/systemctl status *,
/usr/bin/cat /var/log/messages
EOF

# 设置正确权限
chmod 440 /etc/sudoers.d/sudotest

# 验证配置语法
visudo -cf /etc/sudoers.d/sudotest
```

```
# 测试
su - sudotest
sudo systemctl status sshd      # 应该成功，无需密码
sudo systemctl restart sshd     # 应该失败或需要密码
exit
```

步骤 5: sudo 日志审计

```
# sudo 操作会记录在日志中
# CentOS/RHEL
tail -f /var/log/secure

# Ubuntu/Debian
tail -f /var/log/auth.log

# 查看特定用户的 sudo 记录
grep "sudotest" /var/log/secure
```

sudo 常用命令

命令	说明
<code>sudo command</code>	以 root 身份执行命令
<code>sudo -u user command</code>	以指定用户身份执行
<code>sudo -l</code>	查看当前用户的 sudo 权限
<code>sudo -i</code>	切换到 root shell
<code>sudo -s</code>	以 root 身份启动 shell
<code>visudo</code>	安全编辑 sudoers 文件

2.5 实验六：综合实战 – 企业用户权限规划

实验目标

模拟真实企业环境，完成完整的用户权限规划和配置。

场景描述

你是一家互联网公司的运维工程师，需要为新服务器配置用户权限体系：

部门	用户	权限需求
开发部	dev1, dev2	访问 /data/code，可以部署代码
运维部	ops1, ops2	完全 sudo 权限，管理所有服务
测试部	qa1	访问 /data/test，只读生产日志
DBA	dba1	管理数据库相关服务和目录

实验步骤

步骤 1：创建用户组

```
# 创建部门组
groupadd dev
groupadd ops
groupadd qa
groupadd dba

# 验证
cat /etc/group | grep -E "(dev|ops|qa|dba):"
```

步骤 2：创建用户

```
# 创建开发用户
useradd -g dev -c "开发工程师1" dev1
useradd -g dev -c "开发工程师2" dev2

# 创建运维用户
useradd -g ops -G wheel -c "运维工程师1" ops1
useradd -g ops -G wheel -c "运维工程师2" ops2

# 创建测试用户
useradd -g qa -c "测试工程师" qa1

# 创建DBA用户
useradd -g dba -c "数据库管理员" dba1

# 设置密码（生产环境应使用强密码）
echo "Dev@123" | passwd --stdin dev1
echo "Dev@123" | passwd --stdin dev2
echo "Ops@123" | passwd --stdin ops1
echo "Ops@123" | passwd --stdin ops2
echo "Qa@1234" | passwd --stdin qa1
echo "Dba@123" | passwd --stdin dba1
```

步骤 3：创建项目目录结构

```
# 创建目录结构
mkdir -p /data/{code,test,logs,mysql}

# 查看目录结构
tree /data 2>/dev/null || ls -la /data
```

步骤 4：配置目录权限

```
# 代码目录 - 开发组可读写
chown root:dev /data/code
chmod 2775 /data/code # SGID, 新文件继承组

# 测试目录 - 测试组可读写
chown root:qa /data/test
```

```
chmod 2775 /data/test
```

```
# 日志目录 - 所有人可读, 仅 root/ops 可写
```

```
chown root:ops /data/logs
```

```
chmod 755 /data/logs
```

```
# 数据库目录 - 仅 DBA 可访问
```

```
chown root:dba /data/mysql
```

```
chmod 2770 /data/mysql
```

```
# 验证权限配置
```

```
ls -la /data/
```

步骤 5: 配置 sudo 权限

```
# 创建 sudo 配置文件
```

```
cat > /etc/sudoers.d/company << 'EOF'
```

```
# 运维组 - 完全权限 (wheel 组已有, 这里额外配置)
```

```
%ops    ALL=(ALL)    ALL
```

```
# 开发组 - 只能部署代码和查看特定服务状态
```

```
%dev    ALL=(ALL)    NOPASSWD: /usr/bin/systemctl status nginx, \
          /usr/bin/systemctl restart nginx, \
          /usr/bin/systemctl reload nginx, \
          /usr/bin/tail -f /data/logs/*
```

```
# 测试组 - 只能查看日志
```

```
%qa     ALL=(ALL)    NOPASSWD: /usr/bin/tail -f /data/logs/*, \
          /usr/bin/cat /data/logs/*
```

```
# DBA组 - 管理数据库服务
```

```
%dba    ALL=(ALL)    NOPASSWD: /usr/bin/systemctl * mysqld, \
          /usr/bin/systemctl * mariadb, \
          /usr/bin/mysql, \
          /usr/bin/mysqldump
```

```
EOF
```

```
# 设置权限
```

```
chmod 440 /etc/sudoers.d/company
```

```
# 验证语法
visudo -cf /etc/sudoers.d/company
```

步骤 6：测试权限配置

```
# 测试开发用户权限
echo "=== 测试 dev1 权限 ==="
su - dev1 -c "touch /data/code/test.txt && echo '创建代码文件：成功'"
su - dev1 -c "touch /data/test/test.txt 2>/dev/null && echo '创建测试文件：成功' || echo '创建测试文件：失败（预期）'"
su - dev1 -c "sudo -l"

# 测试运维用户权限
echo "=== 测试 ops1 权限 ==="
su - ops1 -c "sudo whoami"
su - ops1 -c "sudo cat /etc/shadow | head -1"

# 测试测试用户权限
echo "=== 测试 qa1 权限 ==="
su - qa1 -c "touch /data/test/qa_test.txt && echo '创建测试文件：成功'"
su - qa1 -c "touch /data/code/test.txt 2>/dev/null && echo '创建代码文件：成功' || echo '创建代码文件：失败（预期）'"

# 测试 DBA 用户权限
echo "=== 测试 dba1 权限 ==="
su - dba1 -c "touch /data/mysql/db_test.txt && echo '创建数据库目录文件：成功'"
su - dba1 -c "touch /data/code/test.txt 2>/dev/null && echo '创建代码文件：成功' || echo '创建代码文件：失败（预期）'"

```

第三部分：课程总结

命令速查表

用户管理命令

命令	功能	常用示例
useradd	创建用户	useradd -g dev -c "描述" username
usermod	修改用户	usermod -aG wheel username
userdel	删除用户	userdel -r username
passwd	设置密码	passwd username
id	查看用户信息	id username
whoami	当前用户名	whoami
su	切换用户	su - username

用户组管理命令

命令	功能	常用示例
groupadd	创建组	groupadd groupname
groupmod	修改组	groupmod -n newname oldname
groupdel	删除组	groupdel groupname
gpasswd	组成员管理	gpasswd -a user group
groups	查看所属组	groups username

权限管理命令

命令	功能	常用示例
<code>chmod</code>	修改权限	<code>chmod 755 file</code>
<code>chown</code>	修改所有者	<code>chown user:group file</code>
<code>chgrp</code>	修改所属组	<code>chgrp group file</code>
<code>ls -l</code>	查看权限	<code>ls -la /path</code>

sudo 相关命令

命令	功能
<code>sudo command</code>	以 root 执行
<code>sudo -l</code>	查看当前 sudo 权限
<code>visudo</code>	编辑 sudoers
<code>sudo -i</code>	切换到 root

课后练习

练习 1：基础操作

- 创建用户 `student`，UID 为 2000，主组为 `students`
- 为该用户设置密码
- 将用户添加到 `wheel` 组

练习 2：权限配置

- 创建目录 `/project`，权限设置为开发组可读写，其他人不可访问
- 测试权限是否生效

练习 3：sudo 配置

1. 配置 `student` 用户可以免密执行 `systemctl status *` 命令
 2. 配置 `student` 用户可以查看 `/var/log/messages` 日志
 3. 验证配置是否正确
-

常见问题解答

Q1: 忘记 root 密码怎么办？

A: 进入单用户模式或救援模式重置密码。

Q2: 为什么 `usermod -G` 会覆盖附加组？

A: 需要使用 `-aG` (append) 才是追加模式。

Q3: SUID 和 sudo 有什么区别？

A: SUID 是程序级别的权限提升，sudo 是命令级别的权限控制，sudo 更灵活、更安全。

Q4: 777 权限为什么危险？

A: 任何人都可以读写执行，可能导致数据被篡改或恶意程序执行。